

Úvod do databázových systémů

Kapitola 8

Transformace E-R schematu do logického modelu

Úvod

- výsledkem konceptuálního modelování je E-R model, který může být dost vzdálený logickému modelu
 - je tedy třeba převést konceptuální model do modelu logického
 - v našem případě budeme převádět E-R model do relačního modelu (resp. nějaké jeho implementace)
 - budeme hledat reprezentaci základních konstruktů konceptuálního modelu
 - RMD obsahuje jediný konstrukt – relaci
 - RMD tedy musí reprezentovat entitní a vztahové typy stejně – schematem relace
 - rekonstrukce E-R modelu z RMD tudíž není obecně jednoznačná

Úvod

- převod ER -> RMD

- jelikož klasickou a nejvíce rozšířenou implementací RMD jsou tabulky relačních databází, budeme převod vysvětlovat rovnou na tabulkách

- rozdíly tabulka vs. relace viz dřívější přednášky
 - abychom byli co nejbližší RMD, předpokládáme dále, že
 - tabulka má definován primární klíč a neobsahuje duplicitní řádky
 - uspořádání řádků tabulky není nijak definováno
 - tabulky v SQL ale umějí oproti relacím i leccos navíc
 - prázdné hodnoty
 - NULL
 - deklarativní integritní omezení
 - NOT NULL, UNIQUE, FOREIGN KEY, aj.

Úvod

- převod ER -> RMD

- převod konceptuálního modelu na relační bývá součástí modelovacích nástrojů (Oracle Data Modeller, Enterprise Architect)
- převod některých konstrukcí konceptuálního modelu může mít několik možných variant
 - např. u ISA hierarchie, vztahu 1:1 apod.
 - optimální varianta pro konkrétní situaci závisí na parametrech, které mohou být mimo konceptuální model (často používané dotazy, způsob uložení dat...)
- někdy nemáme na úrovni RMD efektivní nástroj, jak zajistit kontrolu všech IO z konceptuálního modelu
 - např. povinné členství nedeterminantu ve vztahu 1:N

Základní principy

■ entity

- název entity → název tabulky
- atributy entity → atributy (sloupce) tabulky
 - pro domény atributů entit zvolíme vhodné dat. typy
 - povinný atribut → omezení NOT NULL
 - ident. klíč → omezení PRIMARY KEY
 - alternativní klíče → omezení UNIQUE
- slabá entita → tabulka, jejíž PK obsahuje cizí klíč

■ vztahy

- jediná možnost provázání dat ze dvou tabulek je referenční integrita, cizí klíč (FOREIGN KEY)
- podle kardinality a parciality nutno použít vztahové tabulky a omezení NOT NULL a UNIQUE

Silný entitní typ

- přímočaré – silnému ent. typu odpovídá **schema relace**
 - stejná množina atributů
 - primární klíč odpovídá identifikačnímu klíči entit. typu
 - popisným typům atributů se přiřadí domény
- cíl – normalizovaná tabulka
 - závisí na citu analytika a množství funkčních závislostí (FZ), které uvažujeme
 - mělo by platit, že jediné netriviální FZ odvoditelné z ent. typu jsou závislosti atributů na identifikačním klíči
 - odhalení dalších FZ může indikovat další ent. typ

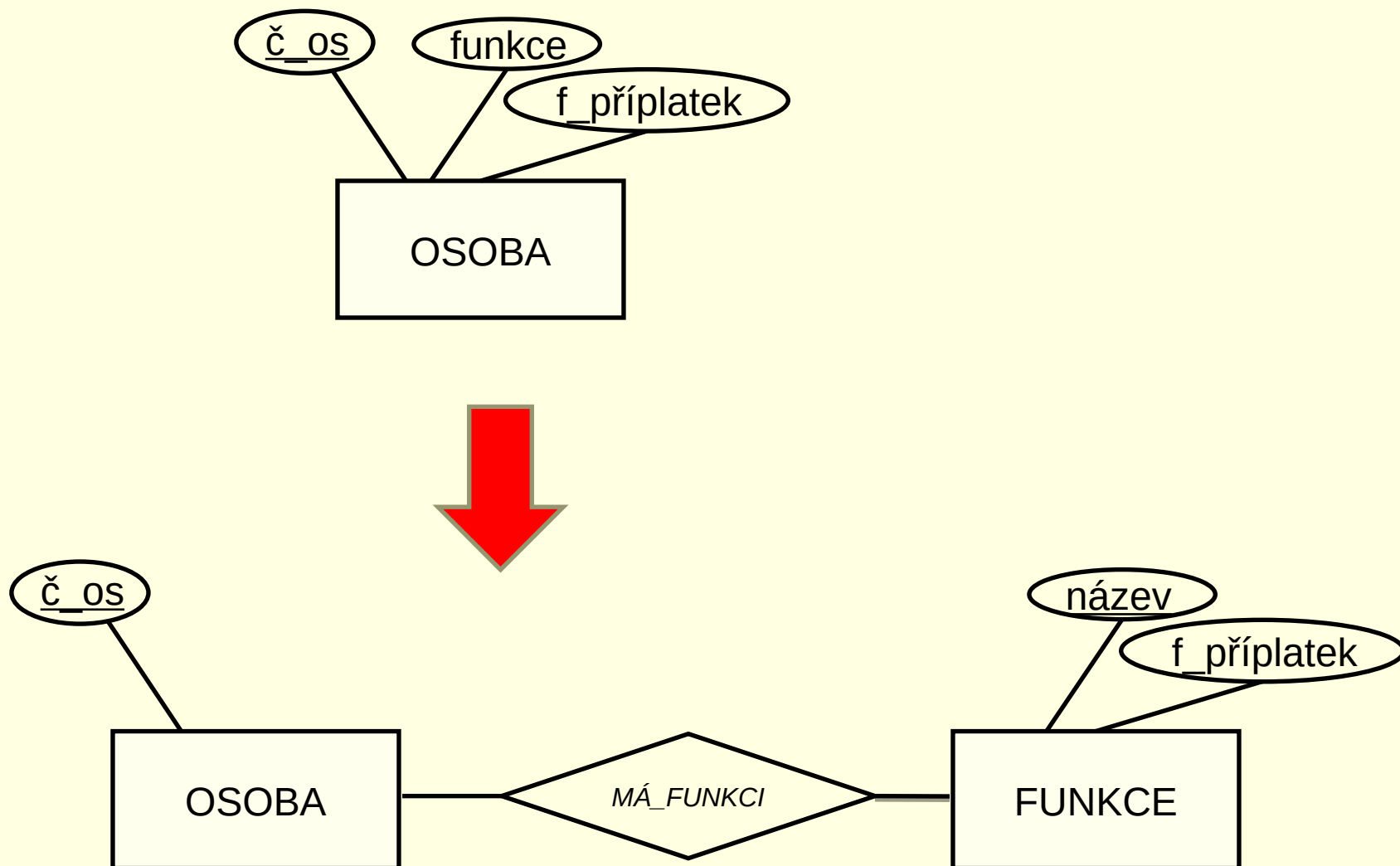
Silný entitní typ – normalizace

- například mějme

OSOBA(č_os, funkce, f_příplatek)

- pokud platí, že funkce → f_příplatek, zavedeme nový entitní typ FUNKCE(název, příplatek) a vztahový typ MÁ_FUNKCI
- tyto problémy je **nutné řešit již na úrovni konceptuálního schématu**

Silný entitní typ – normalizace



Silný entitní typ – normalizace

- jiný příklad

PRACOVNÍK(rč, ..., č_místnosti, č_telefonu)

platí-li: č_telefonu → č_místnosti, není normalizované

- měli bychom zavést dvě entity

- PRACOVNÍK(rč, ..., č_telefonu)

- TELEFONY(č_telefonu, č_místnosti, ...)

- z hlediska uživatele nemusí mít ale tato dekompozice smysl

- lze v SQL částečně řešit pomocí pohledů

Silný entitní typ – příklad

ENTITA

* identifikátor
* povinný_atr
o nepovinný_atr

■ relace:

ENTITA(identifikátor, povinný_atr, nepovinný_atr)

■ SQL:

```
CREATE TABLE Entita (  
    identifikator INT PRIMARY KEY,  
    povinny_atr VARCHAR(20) NOT NULL,  
    nepovinny_atr varchar(40));
```

Silný entitní typ

- vícehodnotové atributy – možná řešení
 - některé SŘBD je přímo umožňují
 - jestliže víme max. počet výskytů atributu, pak pro ně „rezervujeme“ místo v tabulce, nevyužité budou mít například hodnotu NULL
 - zabírá místo
 - zavedeme další tabulku, odpovídající vícehodnotovému atributu a vytvoříme vztah 1:N mezi hlavní tabulkou a novou tabulkou

Silný entitní typ

- skupinové atributy
 - některé SŘBD je přímo umožňují
 - jinak je nutno oželeť hierarchickou strukturu atributu, do tabulky uložíme pouze atomické složky
 - nebo opět využijeme nové tabulky a vztahu

Vztahový typ

■ vztah 1:1

- služební vozidla v podniku
- žádný vůz není využíván více zaměstnanci
- žádný zaměstnanec nevyužívá víc než jeden vůz



- reprezentace závisí na tom, zda je členství ve vztahu povinné či nikoli (parcialita)

Vztahový typ

- vztah 1:1

- **povinné členství pro oba ent. typy**

- atributy obou entitních typů zařadíme do jediné tabulky
 - slévání, přilepení atributů
 - vztah reprezentován implicitně
 - klíčem bude buď č_osoby nebo SPZ (je to jedno)

OSOBA_VŮZ(č_osoby, ... , SPZ, ...)

- SQL

```
CREATE TABLE Osoba_Vuz (  
    č_osoby INT PRIMARY KEY,  
    ...,  
    SPZ CHAR(10) NOT NULL UNIQUE,  
    ...);
```

Vztahový typ

- vztah 1:1

- **povinné členství pro oba ent. typy**

- poznámka: realizace jednou tabulkou nemusí být z hlediska výkonu DB optimální
 - pokud např. často používáme osobu, ale vozidla nikoli, budou atributy vozidla zbytečně zatěžovat systém
 - lze rozdělit na dvě tabulky s **cizím klíčem**

OSOBA(č_osoby, NOT NULL,
VŮZ(SPZ, ..., č_osoby) UNIQUE)

- problémy tohoto řešení
 - pomocí DDL nelze vynutit povinnost členství osoby, nutno řešit např. triggerem nebo v aplikaci
 - nelze odlišit od vztahu 1:1 s nepovinným čl. jedné ent.

Vztahový typ

- vztah 1:1

- **povinné členství pouze pro jeden ent. typ (vůz)**

- osoba nemusí mít auto

- dvě tabulky (VŮZ a OSOBA), do tabulky VŮZ přidáme **cizí klíč** č_osoby

- klíčem Vozu by mohlo být i č_osoby, je to 1:1

OSOBA(č_osoby, ...)

VŮZ(SPZ, ..., č_osoby)

NOT NULL,
UNIQUE

Vztahový typ

- vztah 1:1
 - **povinné členství pouze pro jeden ent. typ (vůz)**
 - SQL

```
CREATE TABLE Osoba (
    č_osoby INT PRIMARY KEY,
    ...);
```

```
CREATE TABLE Vuoz (
    SPZ CHAR(10) PRIMARY KEY,
    ...,
    č_osoby INT NOT NULL UNIQUE
    REFERENCES Osoba(č_osoby));
```

Vztahový typ

■ vztah 1:1

■ **nepovinné členství pro oba ent. typy**

- nelze přilepit atributy ani k vozu, ani k osobě
 - museli bychom jediné připustit prázdné hodnoty (v SQL lze)
- vytvoříme třetí (vztahovou) relaci UŽÍVÁ
 - atributy odpovídající identifikačním klíčům obou ent. typů
 - klíčem nové relace může být č_osoby nebo SPZ

OSOBA(č_osoby, ...)

VŮZ(SPZ, ...)

UŽÍVÁ(č_osoby, SPZ)



NOT NULL,
UNIQUE

- toto řešení není příliš praktické – zbytečné drobení databáze, nutnost mnoha spojení

Vztahový typ

- vztah 1:1

- **nepovinné členství pro oba ent. typy**

- lépe řešit dvěma tabulkami (VŮZ a OSOBA), do tabulky VŮZ přidáme cizí klíč č_osoby, u něhož povolíme NULL

OSOBA(č_osoby, ...)

VŮZ(SPZ, ..., č_osoby)

UNIQUE

Vztahový typ

- vztah 1:1
 - **nepovinné členství pro oba ent. typy**
 - SQL (dvě tabulky)

```
CREATE TABLE Osoba (  
    č_osoby INT PRIMARY KEY,  
    ...);
```

```
CREATE TABLE Vuз (  
    SPZ CHAR(10) PRIMARY KEY,  
    ...,  
    č_osoby INT UNIQUE REFERENCES Osoba(č_osoby));
```

Vztahový typ

- co když má vztah atributy?
 - atributy vztahu se přilepí vždy tam, kde jsou klíče obou souvisejících relací (např. č_osoby i SPZ)
 - buď do té jediné tabulky
 - nebo do tabulky s cizím klíčem (u dvou tabulek)
 - nebo do vztahové tabulky (u tří tabulek)

Vztahový typ

- vztah 1:N
 - PACIENT je determinantem ent. typu POKOJ, opačně to neplatí
 - hraje roli pouze parcialita determinantu
 - dvě možná řešení – dvě nebo tři tabulky
 - v praxi se použijí ponejvíce vždy dvě tabulky a cizí klíč
 - cizí klíč bude mít povoleno nebo zakázáno NULL



Vztahový typ

- vztah 1:N

- **povinné členství determinantu vztahu**

- evidujeme pouze hospitalizované pacienty
 - přilepíme atribut č_pokoje k relaci PACIENT (tj. k determinantu)

PACIENT(rč, ..., č_pokoje)

POKOJ(č_pokoje, ...)



NOT NULL

Vztahový typ

- vztah 1:N
 - **povinné členství determinantu vztahu**
 - SQL

```
CREATE TABLE Pokoj (  
    č_pokoje INT PRIMARY KEY,  
    ...);
```

```
CREATE TABLE Pacient (  
    rč CHAR(10) PRIMARY KEY,  
    ...,  
    č_pokoje INT NOT NULL REFERENCES  
        Pokoj(č_pokoje));
```


Vztahový typ

■ vztah 1:N

■ **nepovinné členství determinantu vztahu**

- evidujeme i ambulantní pacienty

1. přilepíme atribut č_pokoje k relaci PACIENT jako v předchozím případě a musíme připustit prázdnou hodnotu (možné pouze v SQL)
2. zavedení třetí (vztahové) relace UMÍSTĚN
 - atributy odpovídající identifikačním klíčům obou ent. typů
 - klíčem nové relace je klíč determinantu

PACIENT(rč, ...)

POKOJ(č_pokoje, ...)

UMÍSTĚN(rč, č_pokoje)



NOT NULL

Vztahový typ

- vztah 1:N
 - **nepovinné členství determinantu vztahu**
 - SQL (dvě tabulky)

```
CREATE TABLE Pokoj (  
    č_pokoje INT PRIMARY KEY,  
    ...);
```

```
CREATE TABLE Pacient (  
    rč CHAR(10) PRIMARY KEY,  
    ...,  
    č_pokoje INT REFERENCES Pokoj(č_pokoje));
```

Vztahový typ

- vztah 1:N
 - **nepovinné členství determinantu vztahu**
 - SQL (tři tabulky)

```
CREATE TABLE Pokoj (  
    č_pokoje INT PRIMARY KEY, ...);
```

```
CREATE TABLE Pacient (  
    rč CHAR(10) PRIMARY KEY, ...);
```

```
CREATE TABLE Umisten (  
    rč CHAR(10) PRIMARY KEY REFERENCES Pacient(rč),  
    č_pokoje INT NOT NULL REFERENCES  
        Pokoj(č_pokoje));
```

Vztahový typ

■ vztah M:N

- např. vůz může náležet více zaměstnancům, jeden zaměstnanec může mít přiděleno víc vozů
- vždy tři tabulky (dvě pro entity, jedna pro vztah)
- primární klíč vztahové tabulky
 - závisí na způsobu implementace (viz dále)
 - jeden umělý identifikátor (implementace silnou entitou)
 - dvojice příslušných cizích klíčů (slabá entita bez část. klíče)
 - dvojice cizích klíčů + další částečný klíč (slabá entita)



OSOBA(č_osoby, ...)

VŮZ(SPZ, ...)

POUŽÍVÁ(č_osoby, SPZ)

Vztahový typ

- vztah M:N
 - SQL (slabá entita bez částečného klíče)

```
CREATE TABLE Osoba (  
    č_osoby INT PRIMARY KEY, ...);
```

```
CREATE TABLE Vuz (  
    SPZ CHAR(10) PRIMARY KEY, ...);
```

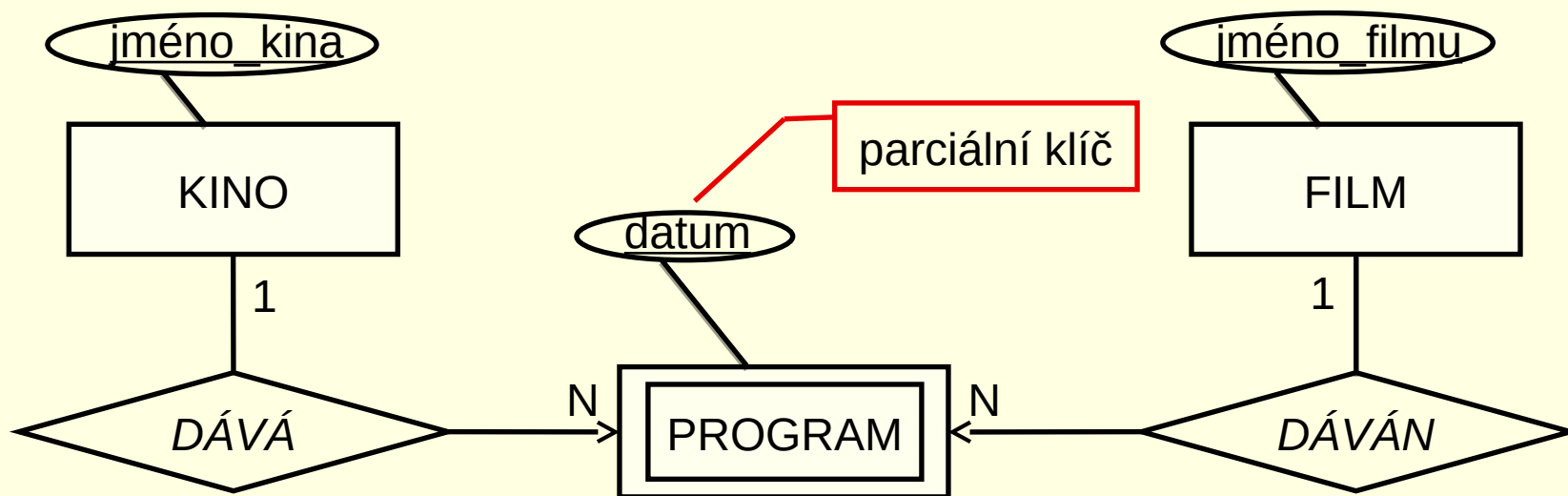
```
CREATE TABLE Uziva (  
    SPZ          CHAR(10) REFERENCES Vuz(SPZ),  
    č_osoby      INT REFERENCES   Osoba(č_osoby),  
    PRIMARY KEY(SPZ, č_osoby));
```

Vztahový typ

- vztah M:N
 - vztah M:N může být ale již v konceptuálním modelu dekomponován
 - pomocí slabé entity bez vlastního částečného klíče
 - viz minulý snímek
 - pomocí slabé entity s dalším částečným klíčem
 - viz další snímky
 - pomocí silné entity s umělým identifikátorem
 - viz další snímky

Vztahový typ

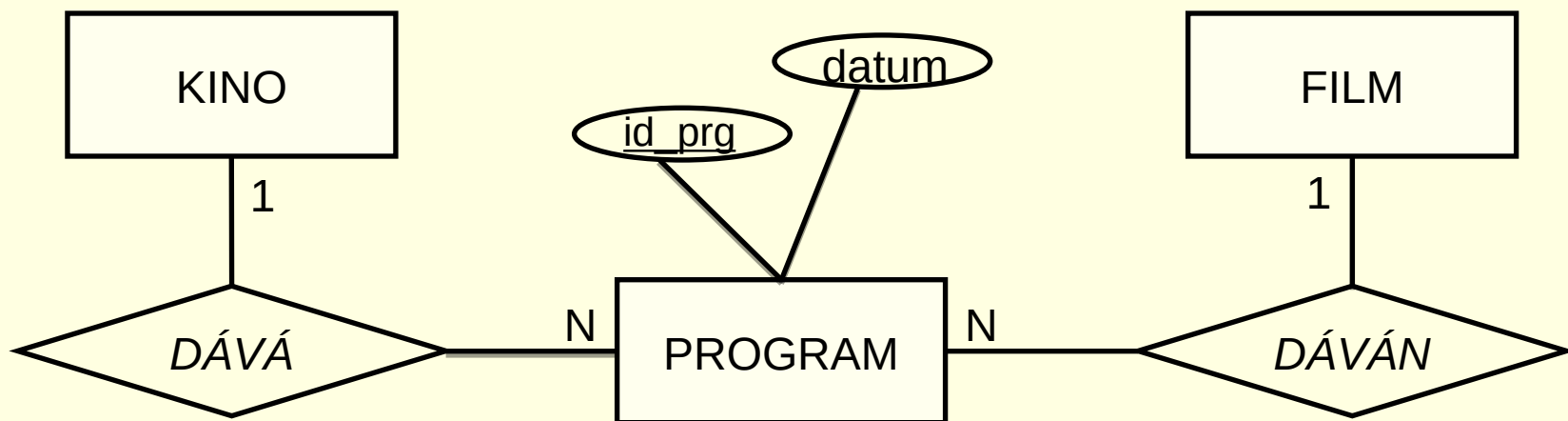
- vztah M:N – dekompozice pomocí slabé entity



```
CREATE TABLE Kino (...); CREATE TABLE Film (...);
CREATE TABLE Program (
    datum DATE,
    nazev_k VARCHAR(30) REFERENCES Kino(nazev_k),
    jmeno_f VARCHAR(30) REFERENCES Film(jmeno_f),
    PRIMARY KEY(datum, nazev_k, jmeno_f) );
```

Vztahový typ

- vztah M:N – dekompozice pomocí silné entity



```
CREATE TABLE Kino (...); CREATE TABLE Film (...);
CREATE TABLE Program (
  id_prg SERIAL PRIMARY KEY,
  datum DATE NOT NULL,
  nazev_k VARCHAR(30) NOT NULL REFERENCES Kino(nazev_k),
  jmeno_f VARCHAR(30) NOT NULL REFERENCES Film(jmeno_f)
);
```


Vztahový typ

- vztah M:N – dekompozice pomocí silné entity
 - výše uvedené řešení ovšem umožňuje zadat do databáze více promítání jednoho filmu v jednom kině v jednom dni
 - pokud chceme zajistit, aby se daný film v daném kině dával max. jednou denně, je třeba dodefinovat ještě unikátnost celé trojice (datum, film, kino)

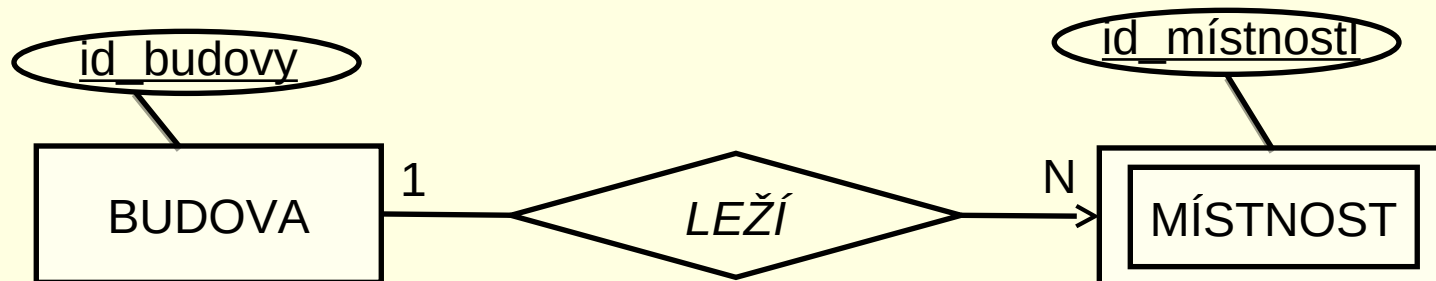
```
CREATE TABLE Program (  
  id_prg SERIAL PRIMARY KEY,  
  datum DATE NOT NULL,  
  nazev_k VARCHAR(30) NOT NULL REFERENCES Kino(nazev_k),  
  jmeno_f VARCHAR(30) NOT NULL REFERENCES Film(jmeno_f),  
  UNIQUE (datum, nazev_k, jmeno_f) );
```

Slabý entitní typ

- identifikační závislost na id. vlastníkoví
 - identifikační vztah je speciálním případem vztahu 1:N, kde slabý entitní typ má povinné členství
 - tedy máme vyřešeno
 - primárním klíčem slabého entitního typu je vždy cizí klíč, odkazující na identifikačního vlastníka, plus případně další (vlastní) **parciální** klíč

Slabý entitní typ

- identifikační závislost s parciálním klíčem



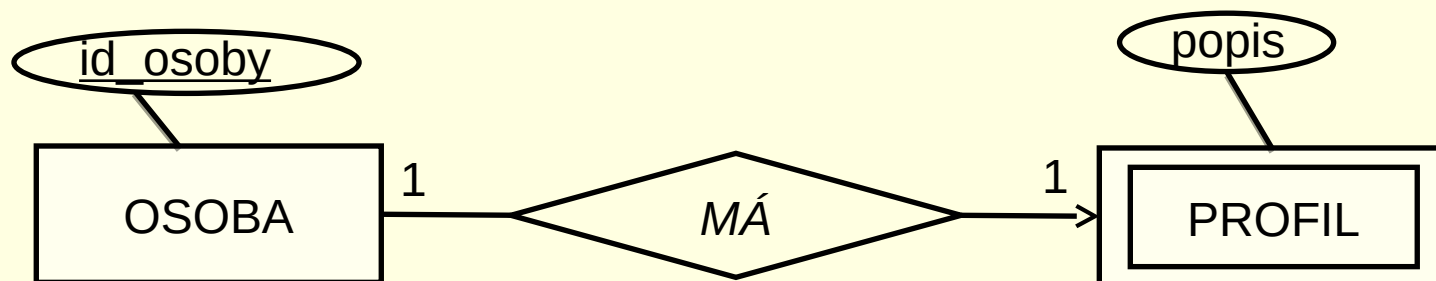
BUDOVA(id_budovy, ...)

MÍSTNOST(id_místnosti, id_budovy, ...)

```
CREATE TABLE Mistnost (  
    id_místnosti INT,  
    id_budovy INT REFERENCES Budova(id_budovy),  
    PRIMARY KEY(id_místnosti, id_budovy) );
```

Slabý entitní typ

- identifikační závislost bez parciálního klíče



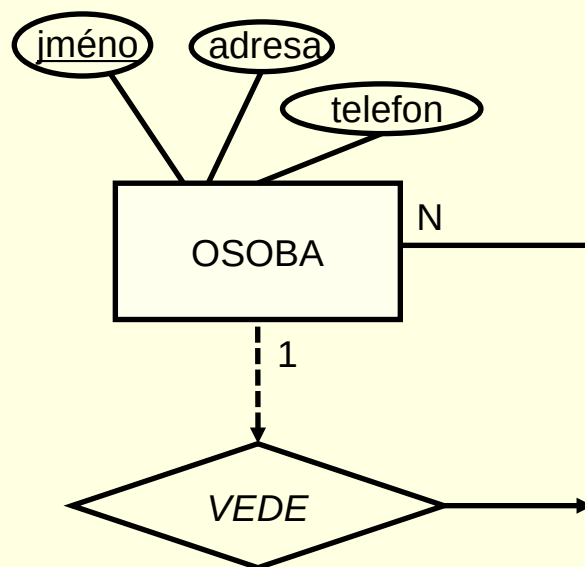
OSOBA(id_osoby, ...)

PROFIL(id_osoby, popis, ...)

```
CREATE TABLE Profil (  
    id_osoby INT PRIMARY KEY  
        REFERENCES Osoba(id_osoby),  
    popis VARCHAR(100) NOT NULL,  
    ..... );
```

Rekurzivní typ vztahu

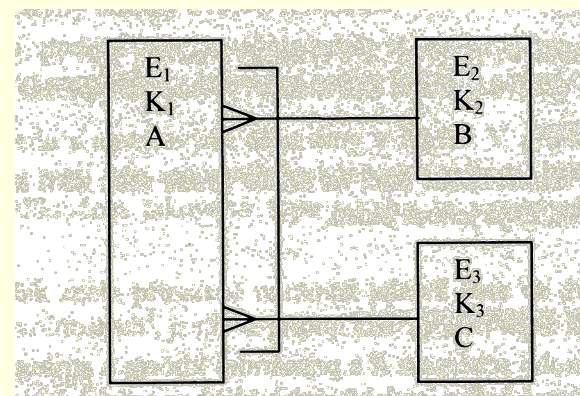
- vztah typu entity sama se sebou
 - spec. případ vztahu 1:N



OSOBA(jméno, adresa, telefon, jméno_vedouci)

```
CREATE TABLE Osoba (  
  jmeno VARCHAR(50) PRIMARY KEY,  
  adresa VARCHAR(150) NOT NULL,  
  telefon CHAR(13) NOT NULL,  
  jmeno_vedouci VARCHAR(50) NOT NULL REFERENCES  
    Osoba(jmeno) );
```

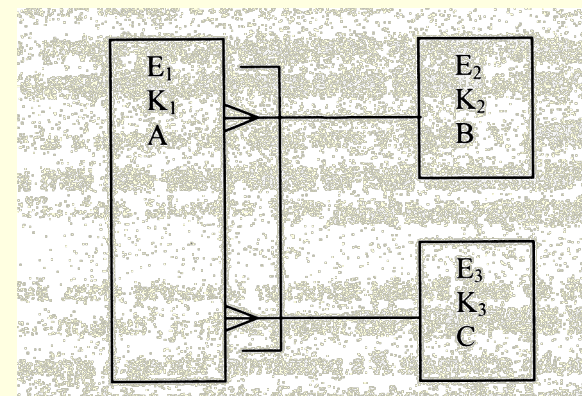
Výlučné vztahy



■ první způsob

- mají-li klíče K2 a K3 stejnou doménu, lze provést
 - $R1(\underline{K1}, id, K, A)$
 - K je cizí klíč odkazující buď na R2 nebo na R3
 - id je atribut, rozlišující, který vztah to je
 - $R2(\underline{K2}, B), R3(\underline{K3}, C)$
- v SQL pak ovšem budeme muset sami ošetřit referenční integritu (třeba triggerem apod.)
 - cizí klíč nemůže být definován tak, že odkazuje jednou tam a podruhé zase jinam

Výlučné vztahy



■ druhý způsob

- obecnější, klíče K2, K3 nemusí mít stejnou doménu
 - $R1(\underline{K1}, K2, K3, A)$
 - nesmí být K2 i K3 současně neprázdné!
 - $R2(\underline{K2}, B), R3(\underline{K3}, C)$
- v SQL musíme pak ošetřit, že bude vyplněn pouze jeden z cizích klíčů – výlučnost

Entitní podtyp (ISA hierarchie)

- narozdíl od slabého typu nemá žádný parciální klíč
 - je identifikován zdrojem ISA hierarchie
- tři způsoby transformace
 - jediná relace se všemi atributy nadtypu i všech podtypů + další atribut rozlišující typ
 - tzv. TPH (table-per-hierarchy) model
 - relace pouze pro entitní podtypy
 - tzv. TPC (table-per-concrete type) model
 - v každé relaci se opakují atributy nadtypu
 - relace pro každý typ (nadtyp i podtypy)
 - tzv. TPT (table-per-type) model
 - odkazy z podtypů do nadtypu

Entitní podtyp (ISA hierarchie)

■ TPH (table-per-hierarchy) model

■ výhody

- přímočaré, rychlé, výhodné pro základní operace (čtení, zápis, update, delete)
- minimalizuje počet tabulek v databázi

■ nevýhody

- mnoho prázdných hodnot (u nevyužitých atributů ostatních podtypů) – riziko porušení integrity, redundance
- porušuje třetí normální formu
- obtížná manipulace s podtypy – nelze je snadno přidávat či odebírat
- u atributů podtypů nelze efektivně vynutit omezení NOT NULL (nutno kontrolovat procedurálně např. triggerem)

Entitní podtyp (ISA hierarchie)

- TPH (table-per-hierarchy) model
 - ošetření integritních omezení
 - nevyužité atributy musí být prázdné
 - kontrola NOT NULL omezení

Entitní podtyp (ISA hierarchie)

- TPT (table-per-type) model

- primární klíče podtypů slouží zároveň jako cizí klíče do nadtypu

- výhody

- zachování integrity jednotlivých tabulek, možnost dodržet třetí normální formu
 - nízká datová redundance
 - snadná manipulace s podtypy

- nevýhody

- pomalejší odezva v dotazech – nutnost častého spojení; počet spojení může být někdy roven počtu podtypů
 - zvýšení počtu tabulek v databázi

Entitní podtyp (ISA hierarchie)

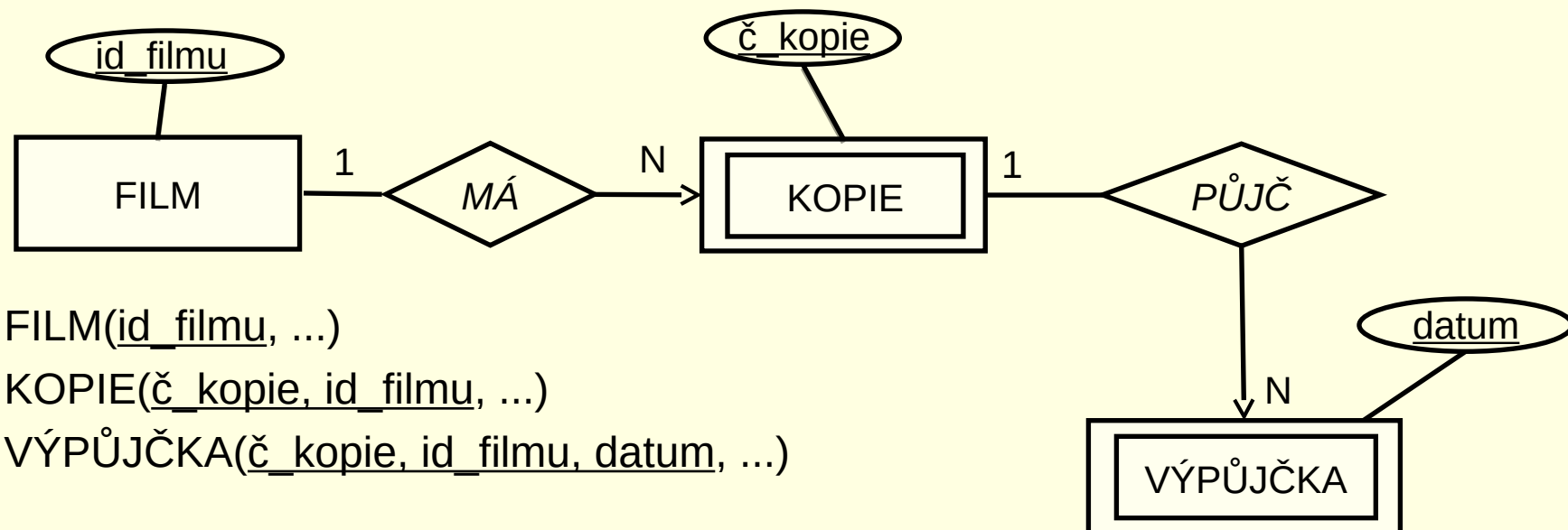
- TPT (table-per-type) model
 - ošetření integritních omezení
 - nutno dbát na disjunktnost podtypů (nesmí se stát, že dvě relace podtypů obsahují tentýž primární klíč)
 - nutno kontrolovat, že pro každou n-tici v relaci nadtypu existuje odpovídající n-tice v právě jedné relaci podtypu
 - pro snadnější čtení dat z příliš složitých struktur je možno převést tuto hierarchii na model TPH pomocí pohledu (VIEW)

Entitní podtyp (ISA hierarchie)

- TPC (table-per-concrete type) model
 - v každé relaci se opakují atributy nadtypu;
 - dílčí relace vlastně „nevědí“ o hierarchii
 - není mezi nimi žádný vztah
 - modifikace struktury hierarchie jsou obtížné
 - mnoho relací obsahuje tytéž atributy, zásahy do nadtypu a jeho vazeb je nutno opakovat ve všech dílčích relacích
 - ošetření integritních omezení
 - některá integritní omezení (např. primární či cizí klíče) musejí být kontrolována nad celou množinou relací
jaksi distribuovaně, je například nutno kontrolovat, aby více dílčích relací neobsahovalo tentýž primární klíč...

Migrace identifikátoru entity

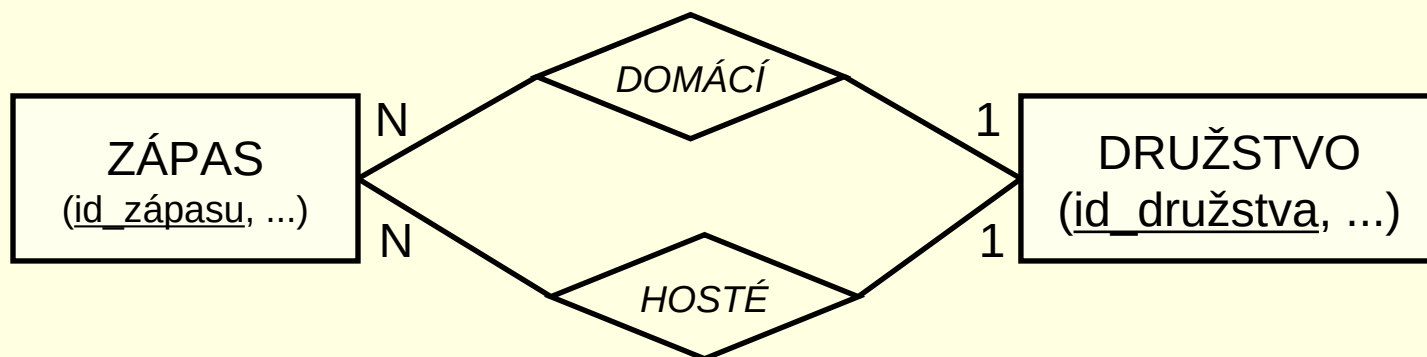
- vznik složeného cizího klíče



```
CREATE TABLE Vypujcka (
  c_kopie INT NOT NULL,
  id_filmu INT NOT NULL,
  datum DATE NOT NULL, ...,
  PRIMARY KEY(c_kopie, id_filmu, datum),
  FOREIGN KEY(c_kopie, id_filmu) REFERENCES Kopie(c_kopie, id_filmu));
```

Cykly a jména rolí

■ mějme schema



■ dva atributy jedné tabulky se nemohou jmenovat stejně

DRUŽSTVO(id_družstva, ...)

ZÁPAS(id_zápasu, ..., **domácí**, **hosté**)

```
CREATE TABLE Zapas (
  id_zapasu SERIAL PRIMARY KEY,
  ...,
  domaci_id_druzstva INT NOT NULL REFERENCES Druzstvo(id_druzstva),
  hoste_id_druzstva  INT NOT NULL REFERENCES Druzstvo(id_druzstva) );
```

Problémy transformace

■ mějme schema



- přidejme vztahu atribut, „do kdy je hospitalizován“

- první možnost

PACIENT(rč, ..., č_pokoje, do_dne)

POKOJ(č_pokoje, ...)

- druhá možnost

PACIENT(rč, ...)

POKOJ(č_pokoje, ...)

UMÍSTĚN(rč, č_pokoje, do_dne)

Problémy transformace

- první možnost neumožňuje korektně evidovat pacienty nezávisle na pokoji (ambulantní)
 - můžeme sice povolit prázdnou hodnotu atributu č_pokoje v relaci PACIENT, ale to vede k potížím
 - musíme zavést další IO, že „má-li číslo pokoje prázdnou hodnotu, pak má prázdnou hodnotu i atribut do_dne“ a případně „má-li číslo pokoje neprázdnou hodnotu, pak má neprázdnou hodnotu i atribut do_dne“
 - pokud bude identifikátor nedeterminantu víceatributový, pak musíme hlídat současné prázdné či neprázdné hodnoty všech identifikačních atributů
- i druhá možnost potřebuje správně definovat referenční integritu

Problémy transformace

- referenční integrita

- týká se téměř všech transformačních operací

- vztahy

- musí vždy platit, že hodnota cizího klíče (v samostatné vztahové entitě nebo klíče přilepeného k determinantu) je obsažena jako hodnota primárního klíče ve vztahované entitě

- např. hodnota atributu č_pokoje přilepeného do relace PACIENT musí existovat jako PK nějaké n-tice v relaci POKOJ

- pozor na složené cizí klíče s povolenou NULL hodnotou

- buď musí být celý prázdný nebo celý korektně vyplněný!

Problémy transformace

- referenční integrita
 - slabé entitní typy
 - musí vždy platit, že hodnota cizího klíče slabé entity je obsažena jako hodnota primárního klíče v relaci identifikačního vlastníka
 - tj. nesmí se stát, že slabá entita odkazuje na neexistující záznam v relaci identifikačního vlastníka

Problémy transformace

- referenční integrita
 - entitní podtypy
 - musí vždy platit, že hodnota primárního klíče entitního podtypu je obsažena jako hodnota primárního klíče v relaci zdroje ISA hierarchie
 - tj. nesmí se stát, že entitní podtyp odkazuje na neexistující záznam v relaci zdroje ISA hierarchie

Problémy transformace

- problémy s parcialitou
 - v reprezentaci vztahu 1:N nejsme schopni odlišit (a zajistit) povinné/nepovinné členství nedeterminantu
 - struktura vždy odpovídá nepovinnému členství nedeterminantu, povinnost nutno vynutit procedurálně
 - v reprezentaci vztahu M:N nejsme schopni odlišit a zajistit parcialitu ani u jedné entity
 - struktura vždy odpovídá nepovinnému členství obou entit, případné povinnosti nutno vynutit procedurálně
- při povinném členství všech účastníků vztahu může nastat problém s aktualizacemi a vkládáním relací
 - deadlock

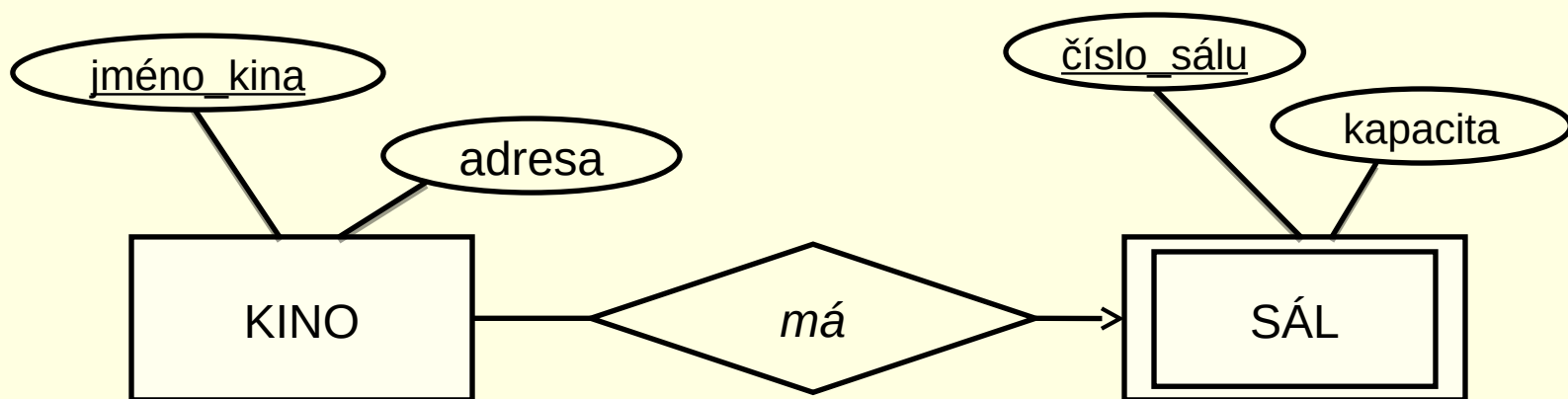
Problémy transformace

- problémy s kardinalitou
 - jak poznat vztah 1:N s povinným členstvím determinantu od vztahu 1:1 s nepovinným členstvím jedné entity?
 - v obou případech řešíme transformaci dvěma tabulkami a přilepením cizího klíče do druhé tabulky, schemata tabulek tedy vypadají stejně
 - u vztahu 1:1 musíme zajistit, aby hodnoty cizího klíče byly pro každou n-tici unikátní; to lze provést buď tak, že cizí klíč bude zároveň i primární klíč (což není u vztahu 1:1 principiálně problém), nebo to lze např. v SQL nařídit (constraint UNIQUE)

Častá chyba

■ častá chyba

- záměna pojmů konceptuálního a logického modelu
- entitě přiřadíme atribut, který slouží k reprezentaci vztahu v relačním modelu:



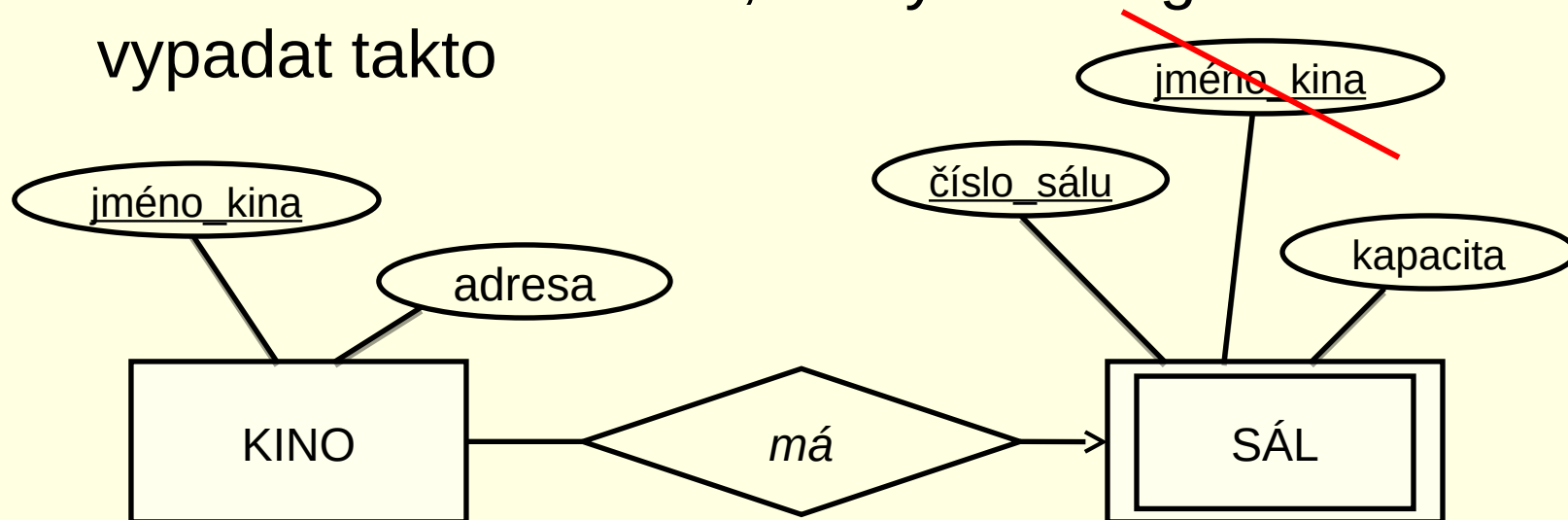
toto schema bude tranformováno do dvou relací:

KINO(jméno_kina, *adresa*)

SÁL(číslo_sálu, jméno_kina, *kapacita*)

Častá chyba

- to ovšem neznamená, že by ER diagram měl vypadat takto



- atribut „jméno_kina“ v entitě SÁL je na konceptuální úrovni **nesmyslný**
 - to, že daný sál patří do daného kina je **jednoznačně řečeno vztahem *má***
 - jakmile jsou v obrázku cizí klíče, už nejde o model konceptuální, ale o grafickou podobu modelu relačního